

Week 3 Module: Programming practice

- Language Used: Java

1. Question

A university's Training and Placement Cell have successfully completed its annual placement drive. The placement department has collected the salary packages (in LPA) offered to students by different companies. However, the records were entered in the order they were received, resulting in an unsorted list of package values.

Before publishing the placement statistics report, the department wants all package values arranged in ascending order. Since the university plans to handle placement data for thousands of students in the coming years, they want a sorting technique that is efficient for large datasets.

Your task is to implement the **Merge Sort algorithm** to sort the package values in increasing order. Merge Sort follows a divide-and-conquer approach by dividing the dataset into smaller subarrays, sorting them recursively, and then merging them back together.

The final sorted list will help the university identify minimum, median, and maximum packages while generating analytical reports.

Input

First line contains an integer **N**, representing the number of placed students.

Second line contains **N space-separated integers**, representing package values (multiplied by 100 to avoid decimals).

Output

Print the package values in ascending order.

Constraints

$$1 \leq N \leq 100000$$

$$100 \leq \text{Package} \leq 10000$$

Logic

1. Start.
2. If the array is null or contains less than 2 elements, return.
3. Find the middle index of the array.
4. Divide the array into two subarrays:
 - Left subarray from index 0 to mid - 1.
 - Right subarray from index mid to the last index.
5. Recursively apply Merge Sort on the left subarray.
6. Recursively apply Merge Sort on the right subarray.
7. Merge the two sorted subarrays:
 - Initialize three indices i, j, and k to 0.
 - Compare the current elements of both subarrays.
 - Copy the smaller element into the original array.
 - Increment the corresponding index and repeat until one subarray is exhausted.
8. Copy the remaining elements of the left subarray, if any, into the original array.
9. Copy the remaining elements of the right subarray, if any, into the original array.
10. The original array is now sorted.
11. Stop.

Source Code

```
import static arrayUtilities.Utilities.*;
import static mergeSort.MergeSort.*;

public static void main(String args[]) {
    try (Scanner scan = new Scanner(System.in)) {

        System.out.print("Enter the number of packages: ");
        int n = scan.nextInt();
        int[] arr = new int[n];
        System.out.println("Enter all packages: ");
        for (int i = 0; i < n; i++) {
            int id = scan.nextInt();
            arr[i] = id;
        }
        mergeSort(arr);
        display(arr);
    }
}
```

```
catch (Exception e) {  
    System.out.println(e.getMessage());  
}  
}
```

```
package mergeSort;  
  
import java.util.Arrays;  
  
public class MergeSort {  
    public static void mergeSort(int[] arr ){  
        if(arr == null || arr.length < 2){  
            return;  
        }  
        int arrLength = arr.length;  
        int arrMid = arr.length/2;  
        int[] arrLeft = Arrays.copyOfRange(arr,0,arrMid);  
        int[] arrRight = Arrays.copyOfRange(arr,arrMid,arrLength);  
  
        mergeSort(arrLeft);  
        mergeSort(arrRight);  
  
        merge(arr, arrLeft, arrRight);  
    }  
  
    public static void merge(int[] arr, int[] leftArr, int[] rightArr){  
        int i=0, j=0, k=0;  
  
        while(i<leftArr.length && j<rightArr.length){  
            if(leftArr[i] <= rightArr[j] ){  
                arr[k] = leftArr[i];  
                i++;  
            }  
            else{  
                arr[k] = rightArr[j];  
                j++;  
            }  
            k++;  
        }  
  
        while(i<leftArr.length){  
            arr[k] = leftArr[i];  
            i++;  
            k++;  
        }  
    }  
}
```

```
}  
while(j<rightArr.length){  
    arr[k] = rightArr[j];  
    j++;  
    k++;  
}  
}
```

Output

```
Enter the number of packages: 6  
Enter all packages:  
450 1200 800 650 300 1500  
300 450 650 800 1200 1500  
  
Process finished with exit code 0
```

1. Question (Medium)

A large e-commerce company tracks the time taken (in minutes) to process customer orders before shipment. Due to varying warehouse workloads, processing times differ significantly from one order to another.

The analytics team wants to study operational efficiency by sorting all processing times and generating statistical information. Since millions of orders are processed every month, the company requires an efficient sorting technique with predictable performance.

Your task is to use **Merge Sort** to arrange all processing times in ascending order.

After sorting, perform the following operations:

- Display the sorted processing times.
- Find the **median processing time**.
- Count the number of orders whose processing time is greater than the median.
- Display the difference between the fastest and slowest processing time.

The solution should efficiently handle large datasets and demonstrate a clear understanding of Merge Sort's divide-and-conquer strategy.

Input

- First line contains integer N.
- Second line contains **N space-separated processing times**.

Output

- Sorted processing times.
- Median processing time.
- Count of orders greater than median.
- Difference between maximum and minimum processing time.

Constraints

$$1 \leq N \leq 100000$$

$$1 \leq \text{Time} \leq 100000$$

Logic

1. Start.
2. Read the number of packages from the user.
3. Create an array to store the package values.
4. Input all package values into the array.
5. Sort the array using the Merge Sort algorithm.
6. Display the sorted array.
7. Find the median:
 - If the number of elements is odd, the median is the middle element.
 - If the number of elements is even, the median is the average of the two middle elements.
8. Count the number of package values that are greater than the median.
9. Find the difference between the maximum and minimum package values.
10. Display the median, the number of packages above the median, and the difference between the maximum and minimum values.
11. Stop.

Source Code

```
import static arrayUtilities.Utilities.*;
import static mergeSort.MergeSort.*;

public static void main(String args[]) {
    try (Scanner scan = new Scanner(System.in)) {

        System.out.print("Enter the number of packages: ");
        int n = scan.nextInt();
        int[] arr = new int[n];
        System.out.println("Enter all packages: ");
        for (int i = 0; i < n; i++) {
```

```

        int id = scan.nextInt();
        arr[i] = id;
    }
    mergeSort(arr);
    display(arr);
    float median = median(arr);
    int count = countGreaterThanMedian(arr);
    int difference = differenceBetweenMaxAndMin(arr);

    System.out.println("The median is: " + median);
    System.out.println("Orders above median: " + count);
    System.out.println("The difference is: " + difference);
} catch (Exception e) {
    System.out.println(e.getMessage());
}
}

```

```

package arrayUtilities;

public class Utilities {
    public static float median(int[] arr){
        if(arr == null || arr.length == 0){
            return 0;
        }
        int arrLen = arr.length;
        if(arrLen % 2 != 0){
            return arr[arrLen / 2] ;
        }
        else{
            return (arr[ arrLen/ 2 - 1 ] + arr[arrLen / 2]) / 2.0f;
        }
    }

    public static int countGreaterThanMedian(int[] arr){
        if(arr == null || arr.length == 0){
            return 0;
        }
        int count = 0;
        float median = median(arr);
        for(int num : arr){
            if(num > median){
                count++;
            }
        }
    }
}

```

```
        return count;
    }

    public static int differenceBetweenMaxAndMin(int[] arr){
        if(arr == null || arr.length == 0){
            return 0;
        }
        int max = arr[arr.length - 1];
        int min = arr[0];

        return max - min;
    }
}
```

```
package arrayUtilities;

public class Utilities {
    public static float median(int[] arr){
        if(arr == null || arr.length == 0){
            return 0;
        }
        int arrLen = arr.length;
        if(arrLen % 2 != 0){
            return arr[arrLen / 2] ;

        }
        else{
            return (arr[ arrLen/ 2 - 1 ] + arr[arrLen / 2]) / 2.0f;
        }
    }

    public static int countGreaterThanOrEqualToMedian(int[] arr){
        if(arr == null || arr.length == 0){
            return 0;
        }
        int count = 0;
        float median = median(arr);
        for(int num : arr){
            if(num > median){
                count++;
            }
        }
        return count;
    }
}
```

```
public static int differenceBetweenMaxAndMin(int[] arr){
    if(arr == null || arr.length == 0){
        return 0;
    }
    int max = arr[arr.length - 1];
    int min = arr[0];

    return max - min;
}
```

Output

```
Enter the number of packages: 7
Enter all packages:
12 25 18 30 15 22 40
12 15 18 22 25 30 40
The median is: 22.0
Orders above median: 3
The difference is: 28
```

2. Question (Easy)

A cloud infrastructure company manages hundreds of servers distributed across different geographical locations. Every hour, each server reports its average response time in milliseconds. The collected data is stored in the order it arrives and is not sorted.

To generate performance reports, the operations team wants the response times arranged from the fastest server to the slowest server. Since the data size can become very large, an efficient in-place sorting technique is required.

Your task is to implement **Quick Sort** to sort the response times in ascending order. Built-in sorting methods are not allowed.

After sorting, the operations team will use the report to identify high-latency servers and optimize system performance.

Input

- First line contains integer **N**.
- Second line contains **N space-separated response times**.

Output

Print the response times in ascending order.

Constraints

$1 \leq N \leq 100000$

$1 \leq \text{Response Time} \leq 10000$

Logic

1. Start the program.
2. Read the number of employees from the user.
3. Read all the salaries into an array.
4. Use an outer loop to iterate through the salaries.
5. Use an inner loop to swap salaries
 - If a salary is greater than the next salary in the array then swap those salaries else continue the loop
6. Repeat the process until array is sorted.
7. End the program.

Source Code

```
import static quickSort.QuickSort.*;

public static void main(String args[]) {
    try (Scanner scan = new Scanner(System.in)) {

        System.out.print("Enter the number of trades: ");
        int n = scan.nextInt();
        int[] arr = new int[n];
        System.out.println("Enter all trade values: ");
        for (int i = 0; i < n; i++) {
            int id = scan.nextInt();
            arr[i] = id;
        }
        quickSort(arr);
        display(arr);
    } catch (Exception e) {
        System.out.println(e.getMessage());
    }
}
```

```

package quickSort;

public class QuickSort {
    public static void quickSort(int[] arr){
        quickSort(arr, 0, arr.length - 1);
    }
    public static void quickSort(int[] arr, int low, int high) {
        if(low < high){
            int pivot = partition(arr,low,high);
            quickSort(arr,low,pivot-1);
            quickSort(arr,pivot+1,high);
        }
    }
    public static int partition(int[] arr, int low, int high) {
        int pivot = arr[high];
        int i = low - 1;
        for (int j = low; j <= high-1; j++) {
            if (arr[j]<pivot){
                i++;
                swap(arr,i,j);
            }
        }
        swap(arr,i+1,high);
        return i+1;
    }
    public static void swap(int[] arr,int i, int j) {
        int temp = arr[i];
        arr[i] = arr[j];
        arr[j] = temp;
    }
}

```

Output

```

Enter the number of trades: 6
Enter all trade values:
120 45 78 200 60 95
45 60 78 95 120 200

Process finished with exit code 0

```

2. Question (Medium)

A financial analytics company receives daily trade-value data from multiple stock exchanges. Analysts use this information to identify high-value trading activity and market trends.

The trade values are received in random order throughout the day. Before analysis can begin, the data must be sorted in descending order.

Your task is to implement **Quick Sort** to arrange the trade values from highest to lowest.

After sorting, perform the following operations:

- Display the sorted trade values.
- Find the **Top 5 highest trade values**.
- Calculate the average of the Top 5 values.
- Count how many trade values are greater than the overall average of all trade values.

This problem evaluates your understanding of partitioning, recursion, and real-world data analysis using Quick Sort.

Input

- First line contains integer N .
- Second line contains **N space-separated trade values**.

Output

- Sorted trade values (descending).
- Top 5 values.
- Average of Top 5 values.
- Count of values greater than overall average.

Constraints

$$5 \leq N \leq 100000$$

$$1 \leq \text{Trade Value} \leq 10^9$$

Logic

1. Start.
2. Read the number of trades from the user.
3. Create an array to store the trade values.
4. Input all trade values into the array.
5. Sort the array in ascending order using the Quick Sort algorithm.
6. Display the sorted array in descending order.
7. Extract the top five highest trade values from the sorted array.
8. Display the top five trade values.
9. Calculate the average of the top five trade values.
10. Display the average of the top five trade values.
11. Calculate the overall average of all trade values.
12. Count the number of trade values greater than the overall average.
13. Display the count of trade values greater than the overall average.
14. Stop.

Source Code

```
import static quickSort.QuickSort.*;
import static utility.Utility.*;

public static void main(String args[]) {
    try (Scanner scan = new Scanner(System.in)) {

        System.out.print("Enter the number of trades: ");
        int n = scan.nextInt();
        int[] arr = new int[n];
        System.out.println("Enter all trade values: ");
        for (int i = 0; i < n; i++) {
            int id = scan.nextInt();
            arr[i] = id;
        }
        quickSort(arr);
        displayDescending(arr);
        displayTopFive(arr);
        float average = averageOfTopFive(arr);
        System.out.println("The average of all trade value is: " + average);
        int count = countOfValuesGreater Than Over All Average(arr);
        System.out.println("Values above overall average: " + count);
    }
}
```

```
} catch (Exception e) {  
    System.out.println(e.getMessage());  
}  
}
```

```
package utility;  
  
public class Utility {  
    public static void displayAscending(int[] arr) {  
        for(int num: arr){  
            System.out.print(num+" ");  
        }  
        System.out.println();  
    }  
    public static void displayDescending(int[] arr) {  
        for(int i = arr.length-1; i >= 0; i--){  
            System.out.print(arr[i]+" ");  
        }  
        System.out.println();  
    }  
    public static int[] topFive(int[] arr) {  
        int size = Math.min(arr.length,5);  
        int[] arrTopFive = new int[size];  
        int count = 0;  
        for(int i = arr.length-1; i >= 0 && count < size; i--){  
            arrTopFive[count] = arr[i];  
            count++;  
            if(count == 5){  
                break;  
            }  
        }  
        return arrTopFive;  
    }  
    public static void displayTopFive(int[] arr) {  
        displayAscending(topFive(arr));  
    }  
    public static float averageOfTopFive(int[] arr) {  
        int[] topFive = topFive(arr);  
        int sum = 0;  
        for (int j : topFive) {  
            sum += j;  
        }  
        return (float)sum/topFive.length;  
    }  
    public static int countOfValuesGreaterThanOrEqualToAllAverage(int[] arr) {  
        int count = 0;
```

```

float average = average(arr);
for (int j : arr) {
    if (j > average) {
        count++;
    }
}
return count;
}
public static float average(int[] arr) {
    int sum = 0;
    for(int num: arr){
        sum += num;
    }
    return (float)sum/arr.length;
}
}

```

```

package quickSort;

public class QuickSort {
    public static void quickSort(int[] arr){
        quickSort(arr, 0, arr.length - 1);
    }
    public static void quickSort(int[] arr, int low, int high) {
        if(low < high){
            int pivot = partition(arr,low,high);
            quickSort(arr,low,pivot-1);
            quickSort(arr,pivot+1,high);
        }
    }
    public static int partition(int[] arr, int low, int high) {
        int pivot = arr[high];
        int i = low - 1;
        for (int j = low; j <= high-1; j++) {
            if (arr[j]<pivot){
                i++;
                swap(arr,i,j);
            }
        }
        swap(arr,i+1,high);
        return i+1;
    }
    public static void swap(int[] arr,int i, int j) {
        int temp = arr[i];
        arr[i] = arr[j];
    }
}

```

```
        arr[j] = temp;
    }
    public static void display(int[] arr) {
        for (int num : arr) {
            System.out.print(num + " ");
        }
        System.out.println();
    }
}
```

Output

```
Enter the number of trades: 8
Enter all trade values:
500 1200 700 2000 900 1500 3000 2500
3000 2500 2000 1500 1200 900 700 500
Top 5: 3000 2500 2000 1500 1200
The average top 5 trade value is: 2040.0
Values above overall average: 3

Process finished with exit code 0
```

1.Question (Easy)

An online gaming platform is hosting a national-level tournament involving thousands of participants. At the end of the tournament, player scores are collected from multiple game servers and stored in random order.

Before announcing the final rankings, the organizers need all scores sorted in ascending order. Since the leaderboard data can be very large, they have decided to use **Heap Sort**, which provides guaranteed $O(N \log N)$ performance.

Your task is to implement Heap Sort and arrange all player scores in increasing order.

The sorted output will help tournament officials generate rankings and determine qualification thresholds for future events.

Input

- First line contains integer N.

- Second line contains **N space-separated player scores**.

Output

Print the scores in ascending order.

Constraints

$1 \leq N \leq 100000$

$0 \leq \text{Score} \leq 1000000$

Input Format

- First line contains N
- Second line contains priority scores
- Output Format
- Print sorted priority scores.

Logic

1. Start.
2. Read the number of scores from the user.
3. Create an array to store the scores.
4. Input all the scores into the array.
5. Convert the array into a Max Heap.
6. Repeat until the heap contains only one element:
 - Swap the root element (largest element) with the last element of the heap.
 - Reduce the heap size by one.
 - Restore the Max Heap property by heapifying the root.
7. Stop.

Source Code

```
import static heapSort.HeapSort.*;

public static void main(String args[]) {
    try (Scanner scan = new Scanner(System.in)) {

        System.out.print("Enter the number of scores: ");
        int n = scan.nextInt();
        int[] arr = new int[n];
        System.out.println("Enter all scores: ");
        for (int i = 0; i < n; i++) {
            int score = scan.nextInt();
            arr[i] = score;
        }
        heapSort(arr);
        display(arr);
    }
}
```



```
} catch (Exception e) {  
    System.out.println(e.getMessage());  
}  
}
```

```
package heapSort;  
  
import java.util.Arrays;  
  
public class HeapSort {  
  
    static void heapify(int[] arr, int n, int i) {  
        int largest = i;  
        int l = 2 * i + 1;  
        int r = 2 * i + 2;  
        if (l < n && arr[l] > arr[largest])  
            largest = l;  
        if (r < n && arr[r] > arr[largest])  
            largest = r;  
        if (largest != i) {  
            int temp = arr[i];  
            arr[i] = arr[largest];  
            arr[largest] = temp;  
            heapify(arr, n, largest);  
        }  
    }  
  
    public static void heapSort(int[] arr) {  
        int n = arr.length;  
        for (int i = n / 2 - 1; i >= 0; i--)  
            heapify(arr, n, i);  
        for (int i = n - 1; i > 0; i--) {  
            int temp = arr[0];  
            arr[0] = arr[i];  
            arr[i] = temp;  
            heapify(arr, i, 0);  
        }  
    }  
  
    public static void display(int[] arr) {  
        for (int num : arr) {  
            System.out.print(num + " ");  
        }  
        System.out.println();  
    }  
}
```

```

    }
}

public void selectionSort(List<Integer> list, int size){
    for(int i = 0; i < size; i++){
        int min = list.get(i);
        for (int j = i+1; j < size; j++){
            if(list.get(j) < min){
                min = list.get(j);
                int temp = list.get(i);
                list.set(i, min);
                list.set(j, temp);
            }
        }
    }
}

public void display(List<Integer> list){
    for (Integer prScore : list) {
        System.out.print(prScore + " ");
    }
}

} catch (Exception e) {
    throw new RuntimeException(e);
}
}
}

```

Output

```

Enter the number of scores: 6
Enter all scores:
850 1200 950 600 1500 1000
600 850 950 1000 1200 1500

Process finished with exit code 0

```

2.Question (Medium)

A multi-specialty hospital maintains records of emergency response times for all critical cases handled during a month. Each response time represents the number of minutes taken by the emergency team to reach and begin treatment for a patient.

Hospital administrators want to analyze the efficiency of their emergency services. To do so, all response times must first be sorted using **Heap Sort**.

After sorting the response times in ascending order, generate the following statistics:

- Display the sorted response times.
- Find the fastest response time.
- Find the slowest response time.
- Calculate the average response time.
- Count the number of cases handled faster than the average.
- Calculate the percentage of cases handled faster than the average.

The hospital intends to use this information to improve resource allocation and reduce emergency response delays.

Input

- First line contains integer N.
- Second line contains **N space-separated response times**.

Output

- Sorted response times.
- Fastest response time.
- Slowest response time.
- Average response time.
- Number of cases faster than average.
- Percentage of cases faster than average.

Constraints

$$1 \leq N \leq 100000$$

$$1 \leq \text{Time} \leq 500$$

Logic

1. Start.
2. Read the number of scores from the user.
3. Create an array to store the scores.
4. Input all the scores into the array.
5. Convert the array into a Max Heap.
6. Repeat until the heap contains only one element:

- Swap the root element (largest element) with the last element of the heap.
 - Reduce the heap size by one.
 - Restore the Max Heap property by heapifying the root.
7. Display the sorted array.
 8. Find the fastest response (minimum score).
 9. Find the slowest response (maximum score).
 10. Calculate the average of all scores.
 11. Count the number of scores faster than the average.
 12. Calculate the percentage of scores faster than the average.
 13. Display the fastest response, slowest response, average, count, and percentage.
 14. Stop.

Source Code

```
import static heapSort.HeapSort.*;
import static utility.Utility.*;

public static void main(String args[]) {
    try (Scanner scan = new Scanner(System.in)) {

        System.out.print("Enter the number of scores: ");
        int n = scan.nextInt();
        int[] arr = new int[n];
        System.out.println("Enter all scores: ");
        for (int i = 0; i < n; i++) {
            int score = scan.nextInt();
            arr[i] = score;
        }
        heapSort(arr);
        display(arr);
        int fastest = fastestResponse(arr);
        int slowest = slowestResponse(arr);
        float avg = average(arr);
        int count = casesFasterThanAverage(arr);
        float perc = percentageOfCasesFasterThanAverage(arr);
        System.out.println("Fastest Response: " + fastest);
        System.out.println("Slowest Response: " + slowest);
        System.out.println("The average score is: " + avg);
        System.out.println("Cases faster than average: " + count);
        System.out.println("The percent is: " + perc);
    } catch (Exception e) {
        System.out.println(e.getMessage());
    }
}
```

```

package heapSort;

import java.util.Arrays;

public class HeapSort {

    static void heapify(int[] arr, int n, int i) {
        int largest = i;
        int l = 2 * i + 1;
        int r = 2 * i + 2;
        if (l < n && arr[l] > arr[largest])
            largest = l;
        if (r < n && arr[r] > arr[largest])
            largest = r;
        if (largest != i) {
            int temp = arr[i];
            arr[i] = arr[largest];
            arr[largest] = temp;
            heapify(arr, n, largest);
        }
    }

    public static void heapSort(int[] arr) {
        int n = arr.length;
        for (int i = n / 2 - 1; i >= 0; i--)
            heapify(arr, n, i);
        for (int i = n - 1; i > 0; i--){
            int temp = arr[0];
            arr[0] = arr[i];
            arr[i] = temp;
            heapify(arr, i, 0);
        }
    }

    public static void display(int[] arr) {
        for (int num : arr) {
            System.out.print(num + " ");
        }
        System.out.println();
    }

}

public void selectionSort(List<Integer> list, int size){
    for(int i = 0; i < size; i++){
        int min = list.get(i);
        for (int j = i+1; j < size; j++){
            if(list.get(j) < min){
                min = list.get(j);
            }
        }
        int temp = list.get(i);
        list.set(i, min);
        list.set(j, temp);
    }
}

```

```

        list.set(i, min);
        list.set(j, temp);
    }
}
}

public void display(List<Integer> list){
    for (Integer prScore : list) {
        System.out.print(prScore + " ");
    }
}

} catch (Exception e) {
    throw new RuntimeException(e);
}
}
}

```

```

package utility;

public class Utility {
    public static int fastestResponse(int[] arr){
        return arr[0];
    }
    public static int slowestResponse(int[] arr) {
        return arr[arr.length-1];
    }
    public static float average(int[] arr) {
        int n = arr.length;
        float sum = 0;
        for (int num : arr) {
            sum += num;
        }
        return sum/n;
    }
    public static int casesFasterThanAverage(int[] arr) {
        float avg = average(arr);
        int count = 0;
        for(int num : arr) {
            if(num < avg) {
                count++;
            }
        }
    }
}

```

```
    }  
    return count;  
}  
public static float percentageOfCasesFasterThanAverage(int[] arr) {  
    int n = casesFasterThanAverage(arr);  
    return (float)(n*100)/(float)arr.length;  
}  
}
```

Output

```
Enter the number of scores: 8  
Enter all scores:  
12 7 15 9 20 5 10 8  
5 7 8 9 10 12 15 20  
Fastest Response: 5  
Slowest Response: 20  
The average score is: 10.75  
Cases faster than average: 5  
The percent is: 62.5  
  
Process finished with exit code 0
```